CS324 Coursework Report – RoboRun

Section I.

Instructions: Running the Game Locally

Installation:

- Unzip the game files to your preferred directory.
- Open a terminal or command prompt window.
- Navigate to the game's directory using ***cd path/to/game***.
- If the threejs library is not locally installed within the project directory, then add it
- If it is then skip this step
    - Threejs **r158** works for the game (though the game was developed on the latest version)
    - https://github.com/mrdoob/three.js
    - Add threejs folder to the project directory and rename the folder 'threejs'
    - The game directory should look like this:
        - **Solution/threejs/build, examples, utils**
        - **Solution/assets/images, blender files**
        - **Solution/html, javascript, pdf(report) files**
- The import statements will not work unless you have threejs repo downloaded.

Running the Server:

- Execute ***python3 -m http.server*** to start the local development server at index.html

Accessing the Game:

- Open chrome and go to local server url shown in the console (usually http://localhost:8000).
- You should now be able to play the game in your browser.

Note: the game was developed with the build tool vite, and then reconfigured to work with relative imports to local threejs files. An import map is used in index.html to get the code working correctly. (I am unsure why, but it does not work without this import map).
Therefore, make sure that the version of chrome being used is an updated version as some older versions do not work with import maps.

Have fun!

Section II.

Introduction

RoboRun is a 3D parkour game where you guide a robot across different environments to reach the end of each map. Tackle the Ice Level (Level 1), where you must keep your footing on slippery platforms, and the Fire Level (Level 2), where avoiding lava pools is the objective. The game is controlled through the WASD keys for movement, spacebar to jump, and 'C' to toggle between camera views.

The game is built using Three.js with UI elements created in JavaScript/HTML and styled with CSS. It's designed for easy expansion of levels and includes custom-developed physics for collision detection and gravity. The aim is to create a flexible parkour game that can grow with more levels in the future.

Game Design

Camera Mechanics

The game features a single, dynamic camera that players can toggle between perspectives using the 'C' key, managed by setupCameraToggleListener in commonSetup.js. The camera updates every frame to synchronize with the game's physics, avoiding jitters as it follows the character, implemented in the updateCamera function. The game features two camera perspectives:

- **Third-person view**: Follows behind the character for an immersive experience.
- **Bird's eye view**: Provides a top-down perspective of the game's environment.

The updateCamera function handles the transition between these views, using linear interpolation for smooth movement. In the level files, camera functions are integrated within the game loop, ensuring a seamless visual experience.

Platforms

Platforms are 'grounds' created as Box class instances—players jump across these to progress. Details are in the Technical Development section. Each level sets up platforms using a grounds array, iterating through to add each to the scene with shadow effects, and corresponding textures.

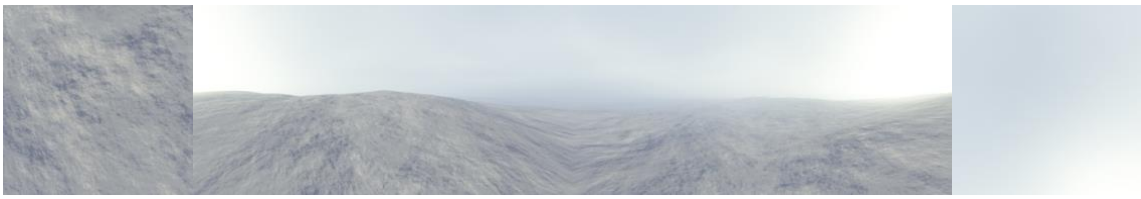
Skyboxes

Skyboxes create a backdrop for the game world, using a textured cube surrounding the play area to depict skies and horizons, adding depth with minimal performance impact. The setup is detailed in commonSetup.js, with textures specified per level and cited in the Reference List.

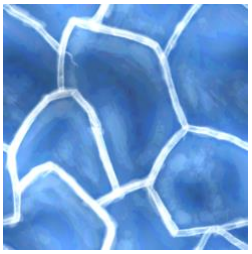
Level 1 Design

Ambient Light: A light blue hue illuminates the entire scene.
Directional Light: Simulates sunlight from behind the character, adding depth.
Hemisphere Light: Adds shine to the lower elements, evoking sunlight reflecting off snow or ice.

Skybox: Icy mountain imagery for environmental backdrop. [1]



Platforms: Ice textures for a coherent thematic appearance, altered to be bluer. [2]



Friction: Velocity on the xz axes was lowered by a friction coefficient rather than immediately set to 0 adding a slipping effect to this level.

Level 2 Design
Ambient Light: Low-intensity light across the scene, maintaining a darker mood to match the theme.
Hemisphere Light: Orange glow on lower elements to enhance the lava-like atmosphere.
Point Lights: Positioned above lava pools to give a glowing effect, without casting shadows for performance optimization.

Skybox: Depicts dark, jagged, and rocky mountain terrain. [3]

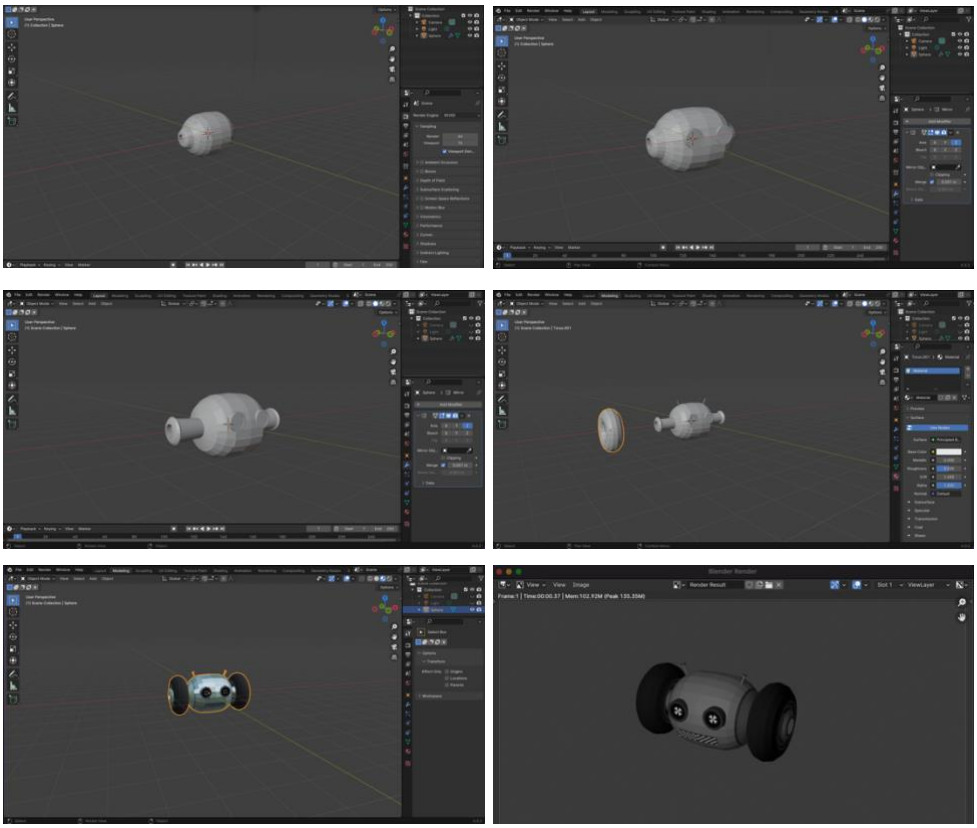
Platforms: Features dark, cracked basalt textures (left) [4]
Lava: Cracked magma textures (right) [5]

Blender design
Blender was used to develop the playable character for the parkour game. The design needed to fit within a single bounding box used in the game for collision detection, so the character had to be relatively boxy.

A robot design with a basic cylindrical body and two round wheels on each side was chosen as the model. The simplicity of this design ensured that the character would fit well within the bounding box, making the game mechanics more consistent and realistic.

Originally, a more complex design for the robot was envisioned, but due to stability issues with Blender on the development computer I was forced to reduce the complexity of the model to ensure a smooth development process. Initially metallic shades were used for coloring the model, but these didn't translate well into the three.js game environment, as the model appeared too dark. Basic colors are applied to the finalized model, which worked better in the game and kept the robot visually appealing.
Overall, the development of the Blender model was a balance between the artistic vision and the technical requirements of the game. The final robot, though simpler than first envisioned, fits well within the game's world and adds a fun element to the gameplay.

Here are some screenshots of the development process:



Technical Development & Implementation

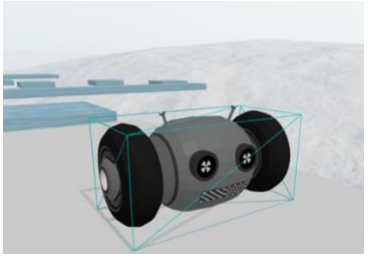
Box Class

The Box class, as defined in gameUtils.js, extends the THREE.Mesh class from the three.js library, specializing in rendering 3D objects. This class serves as a fundamental building block for the game objects. It includes several key features:

- **Geometry and Material**: Utilizes THREE.BoxGeometry for the shape and THREE.MeshStandardMaterial for the appearance, with support for textures and wireframe rendering.

- **Properties**: Includes dimensions (width, height, depth), color, velocity, and position. Additionally, it calculates spatial properties like right, left, bottom, top, front, and back for collision detection.
- **Gravity**: A constant gravity value is set to simulate gravitational pull.
- **Update Methods**: updateSides() recalculates the spatial properties, and update() applies movement and gravity effects. Movement is further divided into applyXZmovement() for horizontal movement and applyYmovement() for vertical movement, incorporating gravity and collision effects.

The Box class concept is derived from the rudimentary three.js collision detection showcased in ChrisCourse's GitHub repository [6], which originally facilitated collision detection solely along the y-axis coupled with basic movement. Our version builds on this foundation, evolving into a comprehensive collision management system that accounts for all faces, ensuring no overlap occurs along the x, y, and z axes for a more sophisticated and reliable interaction.



Wireframe Box around robot

Movement

The movement of the Box object is managed by updating its position based on the velocity vector. This is achieved in two primary functions:

- **applyXZmovement**: Adjusts the x and z positions, handling collisions on these axes. It iterates through potential collision objects (grounds) and applies logic to stop movement upon collision.
- **applyYmovement**: Manages the vertical (y) movement, including the effect of gravity. It detects collision with the ground objects and stops downward movement upon collision with the top face of platforms, otherwise applying gravity to the vertical velocity.

Collision Detection

Collision detection is performed using several utility functions:

- **boxCollision**: Determines if two boxes are colliding in 3D space by checking overlaps along the x, y, and z axes.
- **groundCollision**: Specifically checks for a collision between a Box and a ground object, considering both vertical alignment (using a threshold) and horizontal bounds.
- **sideCollision**: Determines collision on the sides of the box, providing detailed information on which side the collision occurs and whether it's within the bounds of another object on the x and z axes.

Gravity

Gravity is implemented as a constant negative acceleration applied to the y component of the velocity vector of the Box object. The applyYmovement method is responsible for applying gravity when the box is not colliding with the top face of a ground object. This creates a realistic falling effect.

Challenge: Syncing Physics with Time Step

The challenge of syncing physics with a time step rather than the frame rate is addressed to provide the same gameplay experience between different computers at different refresh rates. This is achieved using the following approach:

- Fixed Time Step: A constant time step (1/60 seconds) is used to update physics and game logic at a fixed rate, ensuring consistency regardless of frame rate.

- Accumulator: An accumulator accumulates the delta time between frames. When it exceeds the time step, the game logic and physics are updated, and the accumulator is decreased by the time step amount.
- Continuous Rendering: The rendering of the scene and updating of the camera continue independently of the fixed time step, ensuring smooth visual output.
- Resilience to Frame Rate Variations: This method decouples the physics updates from frame rate variations, ensuring consistent game behavior across different hardware capabilities.

In conclusion, the technical implementation of the game combines 3D rendering techniques with basic physics and collision detection algorithms. The fixed time step approach for syncing physics updates ensures a consistent and fair gaming experience. The modular and extensible code structure facilitates future enhancements and modifications.

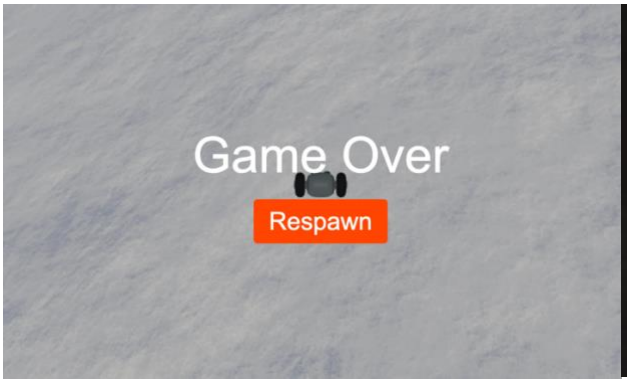
User Interface & Experience

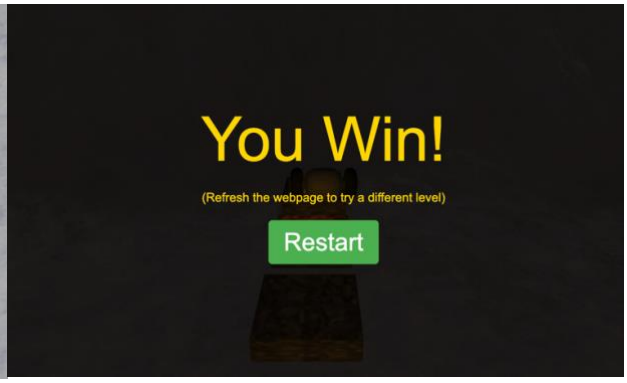
Menu

In developing the game's user interface, the intent was to craft a visually appealing and intuitive experience. The title/menu screen, created in titleScreen.js, showcases a vivid title with dynamic buttons that lead the player into the game and provide essential information like controls and objectives. The design is purposeful, with color cues and text shadows used to enhance aesthetic appeal and user engagement.

Loss/Win Overlays

For transitions between game states, overlays were designed in gameOverlays.js to communicate game outcomes. The 'Game Over' screen prompts a quick respawn, while the winning screen offers a celebratory note with an easy navigation back to the main menu. These elements are not static but react to user interactions, maintaining a fluid experience that keeps the player integrated within the game's environment.





Reference List

[1] Zachery Slochum, SkiingPenguin’s Skybox Pack, Skybox Blizzard
https://opengameart.org/content/skiingpenguins-skybox-pack

[2] Chad Wolfe, Kiira, Iceberg texture
https://opengameart.org/content/hand-painted-texture-iceberg

[3 Zachery Slochum, SkiingPenguin’s Skybox Pack, Skybox Cocoa
https://opengameart.org/content/skiingpenguins-skybox-pack

[4] Cobblestone texture,
https://opengameart.org/node/8038

[5] Lava texture,
https://opengameart.org/node/24158

[6] christopher4lis, ChrisCourses threejs-game,
https://github.com/chriscourses/threejs-game/blob/main/index.html